

Conferencia 2: condicionales y ciclos

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-09-23

La clase pasada terminamos con una calculadora de raíces que funcionaba solo si le dabas los coeficientes correctos. Con $a = 1$, $b = 0$, $c = 1$ se rompía, y con $a = 0$ también. Hoy la arreglamos. Para eso hace falta que el programa **decida** qué hacer según lo que valen sus datos, y eso son los condicionales. Después veremos cómo hacer que un programa **repita** una parte de su trabajo, que son los ciclos. Con esas dos herramientas, al final de la clase vamos a construir un juego de adivinar números.

1. Decidir

Hasta ahora todos nuestros programas se ejecutaban igual siempre: la misma secuencia de instrucciones, de arriba hacia abajo, sin importar los datos. Un condicional rompe esa línea recta y permite que un pedazo de código se ejecute o no.

Valores de verdad

Antes de decidir hay que preguntar, y en Python una pregunta es una expresión como cualquier otra: produce un valor.

```
print(3 < 0)
print(3 > 0)
print(type(3 < 0))
```

```
False
True
<class 'bool'>
```

Ese es el cuarto tipo del curso, después de `int`, `float` y `str`. Se llama `bool`, en honor a George Boole, y tiene exactamente dos valores: `True` y `False`. Se escriben con mayúscula inicial y sin comillas, porque no son texto.

Los operadores que producen valores de verdad son las comparaciones:

OPERADOR	PREGUNTA	EJEMPLO	RESULTADO
<code>=</code>	¿son iguales?	<code>3 = 3</code>	<code>True</code>
<code>≠</code>	¿son distintos?	<code>3 ≠ 3</code>	<code>False</code>
<code><</code>	¿menor?	<code>2 < 3</code>	<code>True</code>
<code>≤</code>	¿menor o igual?	<code>3 ≤ 3</code>	<code>True</code>
<code>></code>	¿mayor?	<code>2 > 3</code>	<code>False</code>
<code>≥</code>	¿mayor o igual?	<code>2 ≥ 3</code>	<code>False</code>

Fíjate en `=`, con dos signos. Un solo `=` es la asignación que vimos la clase pasada, que guarda un valor en una variable. Dos `=` preguntan si dos valores son iguales. Son cosas distintas y confundirlas es el error más común de la semana.

La instrucción if

if ejecuta un bloque de código solo cuando una condición es verdadera:

```
discriminante = -4

if discriminante < 0:
    print("La ecuación no tiene raíces reales")
```

```
La ecuación no tiene raíces reales
```

Hay dos cosas nuevas en la sintaxis. El **dos puntos** al final de la línea del if abre un bloque. La **indentación** de la línea siguiente, cuatro espacios por convención, dice qué instrucciones pertenecen a ese bloque.

Esto es propio de Python y conviene entenderlo bien desde hoy: en la mayoría de los lenguajes la indentación es cosmética y los bloques se delimitan con llaves. En Python la indentación es la sintaxis. Cambiarla cambia el significado del programa, y olvidarla es un error:

```
if 3 < 0:
print("hola")
```

```
File "<stdin>", line 2
    print("hola")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 1
```

El bloque puede tener varias líneas. Lo que está indentado se ejecuta solo si la condición es verdadera; lo que vuelve al margen se ejecuta siempre.

```
x = 7

if x > 5:
    print("x es grande")
    print("bastante grande")

print("fin del programa")
```

```
x es grande
bastante grande
fin del programa
```

Prueba a cambiar x a 3 y observa qué desaparece de la salida y qué no.

else

else da el camino alternativo, el que se toma cuando la condición es falsa. Su bloque también va indentado.

```
discriminante = -4

if discriminante < 0:
    print("No hay raíces reales")
```

```
else:
    print("Hay raíces reales")
```

```
No hay raíces reales
```

Siempre se ejecuta uno de los dos bloques, nunca los dos y nunca ninguno.

elif y los tres casos de la ecuación

La ecuación de segundo grado no tiene dos casos sino tres, según el signo del discriminante: dos raíces distintas, una raíz doble, o ninguna raíz real. Para encadenar varias condiciones está `elif`, que es la contracción de `else if`:

```
import math

a = 1
b = -2
c = 1
discriminante = b ** 2 - 4 * a * c

if discriminante > 0:
    x1 = (-b + math.sqrt(discriminante)) / (2 * a)
    x2 = (-b - math.sqrt(discriminante)) / (2 * a)
    print(f"Dos raíces: {x1} y {x2}")
elif discriminante == 0:
    print(f"Una raíz doble: {-b / (2 * a)}")
else:
    print("No tiene raíces reales")
```

```
Una raíz doble: 1.0
```

Python evalúa las condiciones en orden, de arriba hacia abajo. Ejecuta el bloque de la **primera** que resulte verdadera y se salta todas las demás, incluido el `else`. Solo se ejecuta una rama. Puede haber tantos `elif` como haga falta, y el `else` final es opcional.

Que el orden importe tiene una consecuencia práctica: si escribes primero una condición más general y después una más específica, la segunda nunca se ejecuta.

La trampa de `=` contra `==`

Si escribes un solo `=` dentro de un `if`, Python no te deja ni empezar:

```
discriminante = 0

if discriminante = 0:
    print("raíz doble")
```

```
File "<stdin>", line 3
    if discriminante = 0:
        ^^^^^^^^^^^^^^^^^
SyntaxError: invalid syntax. Maybe you meant '=' or '==' instead of '='?
```

El mensaje es de los buenos: te dice exactamente qué quisiste escribir.

El otro error: cuando a vale cero

Queda el segundo problema de la clase pasada. Si a vale 0 la ecuación no es de segundo grado, y la fórmula divide por $2 * a$, que es cero:

```
import math

a = 0
b = 2
c = -4
discriminante = b ** 2 - 4 * a * c
x1 = (-b + math.sqrt(discriminante)) / (2 * a)
```

```
Traceback (most recent call last):
  File "<stdin>", line 7, in <module>
ZeroDivisionError: division by zero
```

Hay que preguntar por a **antes** de aplicar la fórmula. Y si a es cero, la ecuación es lineal, $bx + c = 0$, con solución $x = -c/b$, salvo que b también sea cero y entonces no hay ecuación ninguna. Eso son condicionales dentro de condicionales, y cada nivel suma cuatro espacios de indentación:

```
a = 0
b = 2
c = -4

if a == 0:
    if b == 0:
        print("Eso no es una ecuación")
    else:
        print(f"Es lineal: x = {-c / b}")
else:
    print("Es de segundo grado")
```

```
Es lineal: x = 2.0
```

La calculadora completa

Juntando todo, la calculadora de la clase pasada queda así, y ya no se rompe con ninguna entrada:

```
import math

a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

if a == 0:
    if b == 0:
        print("Eso no es una ecuación")
    else:
        print(f"Es lineal: x = {-c / b}")
else:
    discriminante = b ** 2 - 4 * a * c

    if discriminante > 0:
        x1 = (-b + math.sqrt(discriminante)) / (2 * a)
        x2 = (-b - math.sqrt(discriminante)) / (2 * a)
        print(f"Dos raíces: x1 = {x1}, x2 = {x2}")
    elif discriminante == 0:
```

```

    print(f"Una raíz doble: x = {-b / (2 * a)}")
else:
    print("No tiene raíces reales")

```

```

$ python raices.py
a: 1
b: 0
c: 1
No tiene raíces reales

$ python raices.py
a: 0
b: 2
c: -4
Es lineal: x = 2.0

```

Mira la forma del programa, no su contenido. La indentación dibuja la estructura de la decisión, y un programa bien indentado se lee como un esquema.

2. Condiciones compuestas

Muchas decisiones necesitan más de una condición. Los operadores and, or y not combinan valores de verdad.

```

edad = 20

print(edad >= 18 and edad < 65)
print(edad < 18 or edad >= 65)
print(not (edad >= 18))

```

```

True
False
False

```

and es verdadero solo si ambos lo son. or es verdadero si al menos uno lo es. not invierte.

p	q	p and q	p or q	not p
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

La precedencia va not, después and, después or, igual que en lógica. Si tienes dudas, usa paréntesis, que además se lee mejor.

Un detalle cómodo de Python: las comparaciones se pueden encadenar como en matemáticas, y significan lo que parece que significan.

```

x = 7

```

```
print(0 <= x <= 10)
print(0 <= x and x <= 10)
```

```
True
True
```

Comparar números reales

Aquí pagamos una deuda de la clase pasada. Dijimos que los `float` son aproximados y que nunca se comparan con `=`. Ahora que tenemos comparaciones, veamos por qué:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

El resultado es matemáticamente correcto y prácticamente inútil. La forma de comparar dos reales es preguntar si están suficientemente cerca, y para eso está `math.isclose`:

```
import math

print(math.isclose(0.1 + 0.2, 0.3))
print(math.isclose(1e-15, 0))
print(math.isclose(1e-15, 0, abs_tol=1e-9))
```

```
True
False
True
```

La segunda línea muestra que comparar contra cero es un caso aparte: `isclose` mide la diferencia relativa al tamaño de los números, y el cero no tiene tamaño. Para compararse contra cero hay que dar una tolerancia absoluta con `abs_tol`.

Esto afecta a nuestra calculadora. `discriminante == 0` es exactamente la comparación que acabamos de decir que no se hace, porque `a`, `b` y `c` vienen de `float(input(...))`. Funciona mientras el usuario escriba números enteros, que se representan exacto, y deja de funcionar con coeficientes que salgan de una medición. La versión rigurosa escribe `math.isclose(discriminante, 0, abs_tol=1e-9)`.

3. Repetir

La calculadora resuelve una ecuación y se muere. Si quieres resolver diez, la ejecutas diez veces. Para que un programa repita una parte de su trabajo está el ciclo.

El ciclo `while`

`while` repite un bloque **mientras** una condición sea verdadera. La sintaxis es la misma del `if`: dos puntos, bloque indentado.

```
n = 5
```

```
while n > 0:
    print(n)
    n = n - 1

print("¡Despegue!")
```

```
5
4
3
2
1
¡Despegue!
```

Todo ciclo while tiene tres partes, y conviene aprender a buscarlas:

1. **Inicializar:** $n = 5$, antes del ciclo, deja la variable en su valor de partida.
2. **Condición:** $n > 0$, se evalúa antes de cada vuelta.
3. **Avanzar:** $n = n - 1$, dentro del bloque, acerca la variable al final.

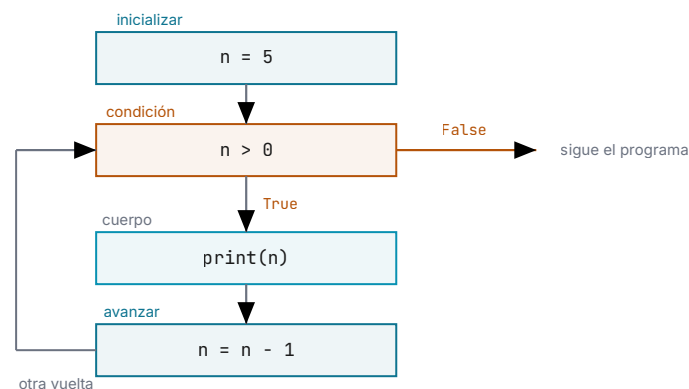


Figura 1. Si falta la tercera parte, la condición nunca deja de ser cierta y el ciclo no termina.

Sigamos la memoria como hicimos la clase pasada. Cada fila es una evaluación de la condición:

VUELTA	n AL ENTRAR	$n > 0$	IMPRIME	n AL SALIR
1	5	True	5	4
2	4	True	4	3
3	3	True	3	2
4	2	True	2	1
5	1	True	1	0
6	0	False	—	—

En la sexta evaluación la condición es falsa, el bloque no se ejecuta y el programa continúa en la línea de después, que imprime ¡Despegue!.

La línea `n = n - 1` es la misma idea de `contador = contador + 1` de la clase pasada: lee el valor actual, calcula el nuevo, lo guarda en la misma variable. Como se escribe todo el tiempo, Python trae una abreviatura para cada operador aritmético:

```
n = 10
n -= 1    # igual que n = n - 1
n += 5    # igual que n = n + 5
n *= 2    # igual que n = n * 2
print(n)
```

28

El ciclo infinito

Si olvidas avanzar, la condición nunca deja de ser verdadera y el programa no termina:

```
n = 5

while n > 0:
    print(n)
```

```
$ python cuenta.py
5
5
5
5
5
5
^C
Traceback (most recent call last):
  File "cuenta.py", line 4, in <module>
    print(n)
KeyboardInterrupt
```

Eso es un ciclo infinito. Se interrumpe con **Ctrl+C**, que le manda una señal al programa y produce el `KeyboardInterrupt` de arriba. Lo vas a hacer muchas veces este curso, así que acostúmbrate: si el programa no responde y la terminal no vuelve al prompt, Ctrl+C.

4. Adivina el número

Ya tenemos las dos herramientas. Vamos a construir un juego con ellas: la computadora piensa un número entre 1 y 100 y tú tienes que adivinarlo.

Números al azar

El módulo `random` de la biblioteca estándar genera números al azar. Se importa igual que `math`:

```
import random

print(random.randint(1, 100))
print(random.randint(1, 100))
print(random.randint(1, 100))
```

```
82
16
59
```

`random.randint(a, b)` devuelve un entero al azar entre `a` y `b`, ambos incluidos. Si ejecutas el programa otra vez, salen otros números.

Paso 1: un solo intento

La primera versión es el condicional de la sección 1 con otra ropa:

```
import random

secreto = random.randint(1, 100)
intento = int(input("Adivina mi número: "))

if intento == secreto:
    print("¡Acertaste!")
else:
    print(f"No. Era {secreto}")
```

Funciona, pero como juego es pésimo: una oportunidad entre cien.

Paso 2: repetir hasta acertar

Metemos la pregunta en un ciclo que repite mientras el intento sea distinto del secreto:

```
import random

secreto = random.randint(1, 100)
intento = 0

while intento != secreto:
    intento = int(input("Adivina mi número: "))

print("¡Acertaste!")
```

La inicialización `intento = 0` merece un comentario. El valor 0 no es un intento del jugador: está ahí porque la condición se evalúa **antes** de la primera vuelta y hace falta que sea verdadera. Se escoge 0 justamente porque está fuera del rango 1 a 100, así que nunca puede coincidir con el secreto por accidente. Es un truco que vas a usar mucho.

Y fíjate en algo: la condición del ciclo es la comparación que antes estaba en el `if`. Condicionales y ciclos no son dos temas pegados con cinta; un ciclo es un condicional que se vuelve a preguntar.

Paso 3: guiar al jugador

Cien intentos a ciegas siguen siendo aburridos. Le decimos al jugador si se pasó o se quedó corto, con un condicional dentro del ciclo:

```
import random

secreto = random.randint(1, 100)
intento = 0

while intento != secreto:
    intento = int(input("Adivina mi número: "))

    if intento < secreto:
```

```

    print("Mi número es mayor")
elif intento > secreto:
    print("Mi número es menor")

print("¡Acertaste!")

```

```

$ python adivina.py
Adivina mi número: 50
Mi número es mayor
Adivina mi número: 75
Mi número es menor
Adivina mi número: 62
Mi número es mayor
Adivina mi número: 68
¡Acertaste!

```

El `elif` no tiene `else`, y está bien: cuando ninguna de las dos condiciones se cumple es porque el jugador acertó, y de eso se encarga la condición del ciclo.

Paso 4: contar los intentos y ponerles un límite

Para contar cuántas veces intentó el jugador hace falta una variable que empiece en cero y suba una unidad por vuelta. Y ya que llevamos la cuenta, démosle siete intentos y ni uno más, lo que convierte la condición del ciclo en una condición compuesta:

```

import random

secreto = random.randint(1, 100)
intento = 0
intentos = 0

while intento != secreto and intentos < 7:
    intento = int(input("Adivina mi número: "))
    intentos += 1

    if intento < secreto:
        print("Mi número es mayor")
    elif intento > secreto:
        print("Mi número es menor")

if intento == secreto:
    print(f"¡Acertaste en {intentos} intentos!")
else:
    print(f"Se acabaron los intentos. Era {secreto}")

```

Aparece algo nuevo al final. El ciclo ahora puede terminar por dos razones distintas, así que después de salir hay que **preguntar por cuál de las dos fue**. Ese `if` de después del ciclo no es un detalle: cada vez que la condición de un `while` tenga un `and`, vas a necesitarlo.

break

Hay otra manera de escribir lo mismo. `break` interrumpe el ciclo en el acto, desde dentro del bloque:

```

import random

secreto = random.randint(1, 100)
intentos = 0

while True:
    intento = int(input("Adivina mi número: "))
    intentos += 1

```

```

if intento == secreto:
    print(f"¡Acertaste en {intentos} intentos!")
    break

if intentos == 7:
    print(f"Se acabaron los intentos. Era {secreto}")
    break

if intento < secreto:
    print("Mi número es mayor")
else:
    print("Mi número es menor")

```

`while True` es un ciclo cuya condición es verdadera siempre, es decir, un ciclo infinito a propósito. Lo que lo termina son los `break`. La versión con `break` hace visible cada razón de salida en el lugar donde ocurre, y evita el `if` de después. La versión anterior pone toda la lógica de terminación en la condición del `while`, donde se lee de un golpe. Ninguna de las dos es mejor siempre; escoge según cuál explique mejor tu programa.

5. Acumuladores y el ciclo contado

El patrón acumulador

Contar los intentos es un caso de algo que se repite en todo el curso: una variable que empieza en un valor neutro y se actualiza en cada vuelta hasta contener el resultado. Se llama **acumulador**.

Sumar los números de 1 a n :

```

n = 10
suma = 0
i = 1

while i <= n:
    suma += i
    i += 1

print(suma)

```

55

Multiplicarlos, que es el factorial:

```

n = 5
producto = 1
i = 1

while i <= n:
    producto *= i
    i += 1

print(producto)

```

120

La única diferencia está en el valor inicial y en la operación. El valor neutro de la suma es 0 y el del producto es 1, porque son los que no alteran el resultado. Empezar el producto en 0 daría 0 siempre, y es un error clásico.

for y range

Los dos ciclos anteriores hacen algo muy común: recorrer los enteros de un rango. Para eso Python tiene una forma más corta:

```
for i in range(5):  
    print(i)
```

```
0  
1  
2  
3  
4
```

range(5) produce los enteros desde 0 hasta 4. Empieza en cero y **termina antes** del número que le das, que es la convención de casi todo en programación y cuesta acostumbrarse. range admite hasta tres argumentos:

FORMA	QUÉ PRODUCE
range(n)	0, 1, ..., n-1
range(a, b)	a, a+1, ..., b-1
range(a, b, p)	a, a+p, a+2p, ... sin llegar a b

```
for i in range(2, 7):  
    print(i, end=" ")  
print()  
  
for i in range(0, 10, 3):  
    print(i, end=" ")  
print()  
  
for i in range(10, 0, -2):  
    print(i, end=" ")  
print()
```

```
2 3 4 5 6  
0 3 6 9  
10 8 6 4 2
```

La suma de 1 a n con for queda así:

```
n = 10  
suma = 0  
  
for i in range(1, n + 1):  
    suma += i
```

```
print(suma)
```

55

Compara las dos versiones. El `for` pone las tres partes del ciclo en una sola línea: inicializa `i`, lo compara y lo avanza por ti. Por eso es imposible olvidar el avance y provocar un ciclo infinito con un `for`. Lo que ganas en seguridad lo pagas en flexibilidad, porque el avance es siempre el mismo.

La regla para escoger es corta: **si sabes cuántas vueltas son, `for`; si depende de algo que pasa dentro del ciclo, `while`**. El juego de adivinar es `while`, porque nadie sabe cuántos intentos va a necesitar el jugador. La suma de 1 a `n` es `for`.

`break` también funciona en un `for`. Y hay un pariente suyo, `continue`, que no sale del ciclo sino que salta a la vuelta siguiente:

```
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i, end=" ")
```

1 3 5 7 9

6. Le toca a la computadora

Invirtamos el juego. Tú piensas un número entre 1 y 100, y la computadora pregunta.

La estrategia obvia es un ciclo que pregunte «¿es 1?», «¿es 2?», «¿es 3?» hasta acertar. Funciona, y en el peor caso pregunta cien veces. La computadora estaría jugando mucho peor que tú, porque tú no ibas probando en orden.

La estrategia buena es la que usa cualquiera: mantener un rango donde el número tiene que estar, y preguntar por su mitad. Cada respuesta descarta la mitad del rango.

```
bajo = 1
alto = 100
preguntas = 0

while bajo < alto:
    medio = (bajo + alto) // 2
    preguntas += 1
    respuesta = input(f"¿Tu número es mayor que {medio}? (s/n) ")

    if respuesta == "s":
        bajo = medio + 1
    else:
        alto = medio

print(f"Tu número es {bajo}, y me tomó {preguntas} preguntas")
```

```
$ python adivina_ella.py
¿Tu número es mayor que 50? (s/n) s
¿Tu número es mayor que 75? (s/n) n
¿Tu número es mayor que 62? (s/n) s
¿Tu número es mayor que 68? (s/n) n
¿Tu número es mayor que 65? (s/n) s
```

```
¿Tu número es mayor que 66? (s/n) s
¿Tu número es mayor que 67? (s/n) n
Tu número es 68, y me tomó 7 preguntas
```

El ciclo termina cuando bajo y alto se juntan, porque entonces solo queda un número posible. Y el rango se encoge a la mitad cada vez.

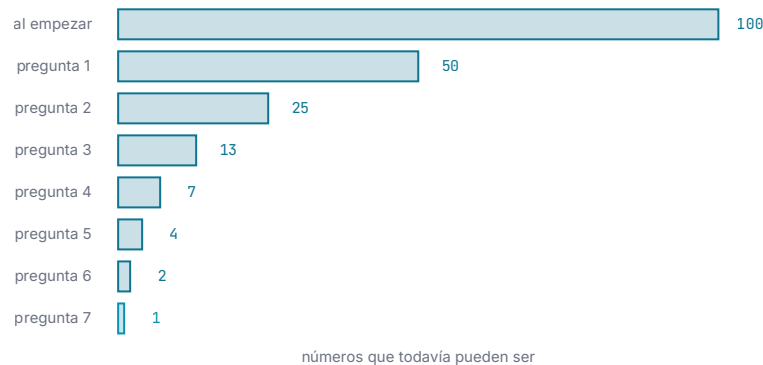


Figura 2. Cada pregunta descarta la mitad de lo que quedaba. Siete preguntas bastan para cien números; con un millón serían veinte.

Siete preguntas en el peor caso, contra cien de la versión anterior. Con un millón de números serían veinte.

Eso se llama **búsqueda binaria**, y es uno de los algoritmos más importantes que vas a aprender. Vuelve, con nombre y apellidos, en el tema de recursión.

7. Resumen

- Una comparación produce un valor de tipo `bool`, que vale `True` o `False`.
- `=` asigna, `==` compara. No son lo mismo.
- `if`, `elif` y `else` escogen un camino. Los dos puntos abren el bloque y la indentación dice qué instrucciones lo forman.
- `and`, `or` y `not` combinan condiciones.
- Dos `float` nunca se comparan con `==`, se comparan con `math.isclose`.
- `while` repite mientras una condición sea verdadera. Tiene tres partes: inicializar, condición y avanzar. Si falta el avance, el ciclo es infinito y se sale con `Ctrl+C`.
- Un acumulador es una variable que empieza en un valor neutro y se actualiza en cada vuelta.
- `for i in range(...)` es la forma corta cuando sabes cuántas vueltas son.
- `break` sale del ciclo y `continue` salta a la vuelta siguiente.

Ejercicios

1. El penúltimo dígito del carné de identidad indica el sexo: par es masculino, impar es femenino. Escribe un programa que lea un carné como número entero e imprima el sexo. No uses `str`: con `//` y `%` alcanza.
2. Escribe un programa que lea tres enteros e imprima qué tipo de triángulo forman: ninguno si no son los lados de un triángulo, escaleno, isósceles o equilátero. Recuerda que cada lado tiene que ser menor que la suma de los otros dos.
3. Escribe un programa que lea un entero e imprima cuántos dígitos tiene, sin usar `str`. Piensa qué le pasa a un número cuando lo divides entre 10.

4. La secuencia de Collatz empieza en un entero n y sigue la regla: si el número es par, se divide entre 2; si es impar, se multiplica por 3 y se le suma 1. Termina al llegar a 1. Escribe un programa que lea n e imprima la secuencia completa y su longitud. Para $n = 17$ la secuencia es 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1. Nadie ha demostrado todavía que este programa siempre termine.
5. Un número es **perfecto** si es igual a la suma de sus divisores propios. Por ejemplo, $28 = 1 + 2 + 4 + 7 + 14$. Escribe un programa que lea un entero y diga si es perfecto. Después imprime todos los números perfectos menores que 10000, y observa cuánto tarda.
6. La sucesión de Fibonacci empieza con 0 y 1, y cada término es la suma de los dos anteriores. Escribe un programa que lea n e imprima el término n -ésimo. Usa un ciclo y dos acumuladores; no hacen falta más.
7. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

(a)

```
for i in range(3):
    for j in range(3):
        if i == j:
            print(i, j)
```

(b)

```
n = 0
while n < 5:
    n += 2
print(n)
```

(c)

```
x = 5
if x > 0:
    print("positivo")
if x > 3:
    print("mayor que tres")
else:
    print("otro")
```

El tercero es el interesante: cámbiale el segundo `if` por un `elif` y explica por qué la salida cambia.